

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

الحمد لله رب العالمين، نحمده ونستعينه ونستغفره والصلاة والسلام على أشرف
الأنبياء والمرسلين نبينا محمد ﷺ



پوهنتون کاردان
KARDAN UNIVERSITY

JavaScript Events

Khalilullah Akbari

k.akbari18@kardan.edu.af

Events and Event Handling

- An event is an action or occurrence detected by the browser.
- JavaScript can listen for an event and run a function in response.
- Common event categories:
 - Mouse events
 - Keyboard events
 - Form and input events
- Event handling connects user actions to program behavior.



Mouse Events

- **click** — fires when an element is clicked.
 - **dblclick** — fires when an element is double-clicked.
 - **mouseover / mouseout** — pointer enters / leaves an element.
 - **mousemove** — pointer moves over an element.
 - **mousedown / mouseup** — mouse button is pressed / released.
-
- Use mouse events to create interactive UI behavior.



Mouse Event Example

1. Mouse over

2. Mouse out

3. Mouse move

```
<button id="btn">Click Me</button>
<div id="box">Move the mouse here</div>

<script>
const btn = document.getElementById("btn");
const box = document.getElementById("box");

btn.addEventListener("click", () => {
  alert("Button clicked!");
});

box.addEventListener("mouseover", () => {
  box.textContent = "Mouse is over the box";
});
</script>
```



Keyboard Events

- **keydown** — fires when a key is pressed.
- **keyup** — fires when a key is released.
- The event object provides useful information such as:
 - e.key → actual key value, e.g. "a", "Enter"
 - e.code → physical key, e.g. "KeyA"
- Useful for shortcuts, search boxes, forms, and games.
- Prefer keydown/keyup; keypress is deprecated.



Keyboard Event Example

1. Keydown: when mouse is down
2. Keyup: when key is leave the button
3. Keypress: at the first when key is getting pressed

```
<input id="inputBox" placeholder="Type something">

<script>
const input = document.getElementById("inputBox");

input.addEventListener("keydown", (e) => {
  console.log("Key:", e.key);

  if (e.key === "Enter") {
    alert("Enter was pressed!");
  }
});
</script>
```

addEventListener()

Syntax:

```
element.addEventListener("event", function);
```

Example:

```
const btn = document.getElementById("btn");

btn.addEventListener("click", () => {
  console.log("First handler");
});

btn.addEventListener("click", () => {
  console.log("Second handler");
});
```

- Multiple listeners can respond to the same event.
- Keeps JavaScript separate from HTML.
- Can later be removed with `removeEventListener()`.



The Event Object

When an event occurs, JavaScript can provide an event object.

Common properties:

- `e.target` — element that triggered the event.
- `e.currentTarget` — element whose listener is running.
- `e.type` — event type, such as "click".
- `e.key` — key pressed in a keyboard event.

Example:

```
button.addEventListener("click", (e) => {  
    console.log(e.target);  
    console.log(e.type);  
});
```



Form and Input Events

- input — fires whenever the user changes the input value.
- change — fires when the value is changed and committed.
- focus — input becomes active.
- blur — input loses focus.
- submit — form is submitted.

Important:

Use the submit event on the <form>, not only the button.

Use `e.preventDefault()` when JavaScript should handle the form.



Basic Form Validation

Goal: check user input before processing the form.

Common checks:

- Required field is not empty.
- Email has a valid basic format.
- Password meets a minimum length.
- Values are within an allowed range.

```
<form id="form">
  <input id="name" placeholder="Name">
  <button type="submit">Submit</button>
</form>

<script>
form.addEventListener("submit", (e) => {
  e.preventDefault();

  if (name.value.trim() === "") {
    alert("Name is required");
    return;
  }

  alert("Form is valid!");
});
</script>
```



Event Bubbling

Event bubbling means an event starts at the target element and then propagates upward through its parent elements.

Example:

If the button is clicked:

1. button handler can run
2. parent handler can run
3. event continues upward

Stop b `e.stopPropagation();`

```
<div id="content">
  <button id="btn">Click Here</button>
</div>

<script>
  let content = document.querySelector("#content")
  let btn = document.querySelector("#btn")

  btn.addEventListener("click", (e) => {
    e.stopPropagation()
    console.log("BTN is clicked.")
  })

  content.addEventListener("click", () => {
    console.log("Div is clicked.")
  })
</script>
```

Event Delegation

Event delegation uses one listener on a parent to handle events from its child elements.

Why use it?

- Fewer event listeners.
- Works well with dynamically added elements.
- Uses event bubbling.

Example:

```
list.addEventListener("click", (e) => {  
  if (e.target.matches("li")) {  
    console.log("Clicked:", e.target.textContent);  
  }  
});
```



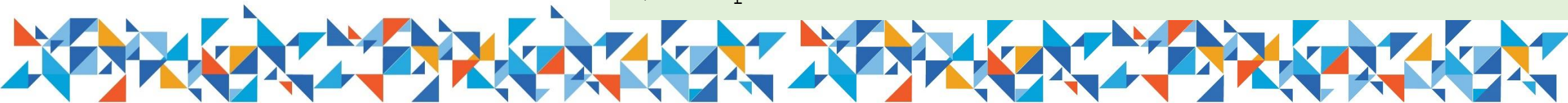
Event Delegation Example

One listener handles all elements.

```
<ul id="list">
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>

<script>
const list = document.getElementById("list");

list.addEventListener("click", (e) => {
  if (e.target.matches("li")) {
    e.target.classList.toggle("selected");
    console.log("Selected:", e.target.textContent);
  }
});
</script>
```



Todo List Example Code

```
<div>
  <input type="text" placeholder="Task..." id="taskInput">
  <div id="taskList"></div>
</div>

<script>
  let taskInput = document.querySelector("#taskInput")
  let taskList = document.querySelector("#taskList")
  taskInput.addEventListener("keyup", (e) => {
    if (e.code == "Enter") {
      // create html element using javascript
      let tp = document.createElement("p")
      tp.textContent = e.target.value
      e.target.value = ""
      taskList.prepend(tp)
    }
  })

  taskList.addEventListener("click", (e) => {
    e.target.remove()
  })
</script>
```



Events: Key Points

- Use `addEventListener()` for modern event handling.
- Know the main event types: mouse, keyboard, and form.
- Use the event object to identify the target and event details.
- Use `submit + preventDefault()` for JavaScript-controlled forms.
- Validate required input before processing data.
- Understand event bubbling before using event delegation.
- Event delegation is useful for lists and dynamic elements.

Practice:

Build a registration form with name, email, and password validation.





پوهنتون کاردان
KARDAN UNIVERSITY

Q & A